



LECTURE TITLE

CHAPTER 27: PROJECT SCHEDULING

Software Engineering

BTech Computer Science and Engineering | Duration: 1 Hour

Software Project Scheduling

Turning Estimates into Timelines

The **Next Critical Step**: We have estimated the effort and cost. Now, we must map that effort against time.

Key Scheduling Questions:

-  Which tasks must come first? (Dependencies)
-  Which tasks can be executed simultaneously? (Parallelization)
-  How do we track our progress? (Monitoring)

The Stakes:

-  A poor schedule guarantees project failure and team burnout.

A good schedule provides a realistic roadmap to successful delivery.

Learning Objectives

What You Will Master

By the end of this lecture, you will be able to:



Understand the Fundamentals: Explain basic scheduling concepts and core principles.



Master the Dynamics of Teams: Understand the complex relationship between adding people and project effort, specifically focusing on Brooks's Law.



Map the Work: Define task sets and properly sequence them by creating task networks.



Visualize the Timeline: Construct and interpret industry-standard time-line charts, such as Gantt charts.



Measure Real Progress: Track a project's actual health using milestone tracking and Earned Value Analysis.

Basic Concepts in Project Scheduling

Why We Schedule

What is Project Scheduling?

The methodical process of assigning specific tasks to distinct time periods and resources.



Why is it Critical?



Provides a clear, actionable roadmap for the entire project team.



Identifies crucial dependencies between tasks before they cause blockers.



Enables quantitative progress tracking against a baseline.



Supports efficient and realistic resource allocation.



Manages and grounds stakeholder expectations in reality.

Key Terminology

Speaking the Same Scheduling Language



Task (Activity)

A distinct unit of work that consumes both time and resources.

Milestone

A significant, zero-duration event marking the completion of major deliverables.

Duration

The actual calendar time required to complete a task from start to finish.

Effort

The actual amount of labor required, usually measured in person-days or person-months.

Dependency

A structural relationship where the start or finish of one task relies on another.

Critical Path

The longest continuous path of dependent tasks through the network; it dictates the absolute minimum project duration.

RULES OF ENGAGEMENT

Seven Basic Principles of Scheduling

Building a Realistic Plan



Compartmentalization

Break the massive project down into discrete, manageable tasks.



Interdependency

Explicitly map out the relationships and sequence between those tasks.



Time Allocation

Estimate the duration for each task based on effort and resource availability.



Effort Validation

Mathematically ensure that the required effort matches the actual person-days your team has available.



Defined Responsibilities

Assign every single task to a specific, named person or team.



Defined Outcomes

Ensure every task produces a tangible, verifiable deliverable.



Defined Milestones

Establish clear checkpoints to measure actual progress against the plan.

People vs. Effort (Brooks's Law)

Why Adding People Makes Late Projects Later

“Adding manpower to a late software project makes it later.”

The Root Cause

Training new people takes immediate time away from productive work, but more importantly, communication overhead grows exponentially with team size.

The Communication Path Formula

$$\text{Paths} = n(n-1)/2$$

(Where n is the number of team members)

Team Size	Communication Paths
3	3
5	10
10	45
20	190

The True Cost of Rushing

The Putnam-Norden-Rayleigh Curve

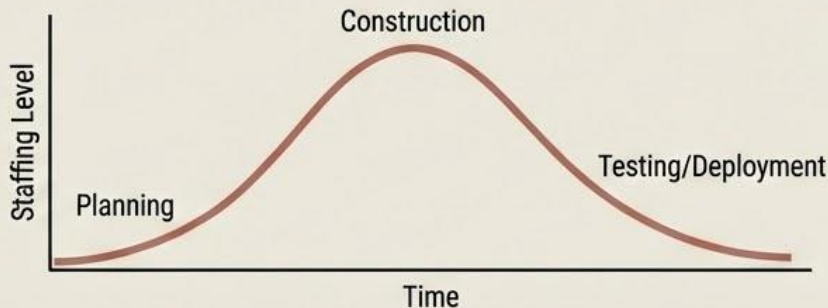
$$\text{Effort} = \text{Size} / \text{Duration}^{(4/3)}$$

The Implication

Because Duration is raised to the power of $4/3$ in the denominator, artificially compressing a project's schedule requires a massive, non-linear increase in effort.

Optimal Staffing (The Rayleigh Curve)

You cannot effectively dump 50 developers onto a project on Day 1. Optimal staffing dictates starting small during planning, ramping up significantly during construction, and decreasing gradually during testing and deployment.



Where Does the Time Go?

Typical Effort Distribution

Allocating the Workload

Based on historical industry data, effort is typically distributed across these primary activities:



Key Takeaway:

Testing and quality assurance often consume as much (or more) effort as the actual writing of the code.

Defining a Task Set

Structuring the Project Scope

What is a Task Set?

A tailored collection of tasks, milestones, and deliverables scaled specifically to the type and size of your project.

Example Task Set (Small Project):

Activity Phase	Key Tasks	Tangible Deliverable
Communication	Meet with stakeholders, document requirements	Requirements Document
Planning	Define tasks, estimate effort, create schedule	Project Plan
Modeling	Create design diagrams, review with team	Design Document
Construction	Code modules, conduct unit tests	Working Code
Deployment	Integrate, test, deliver, train users	Deployed System

Granularity is Key

Refinement of Software Actions.

Breaking High-Level Tasks into Executable Steps

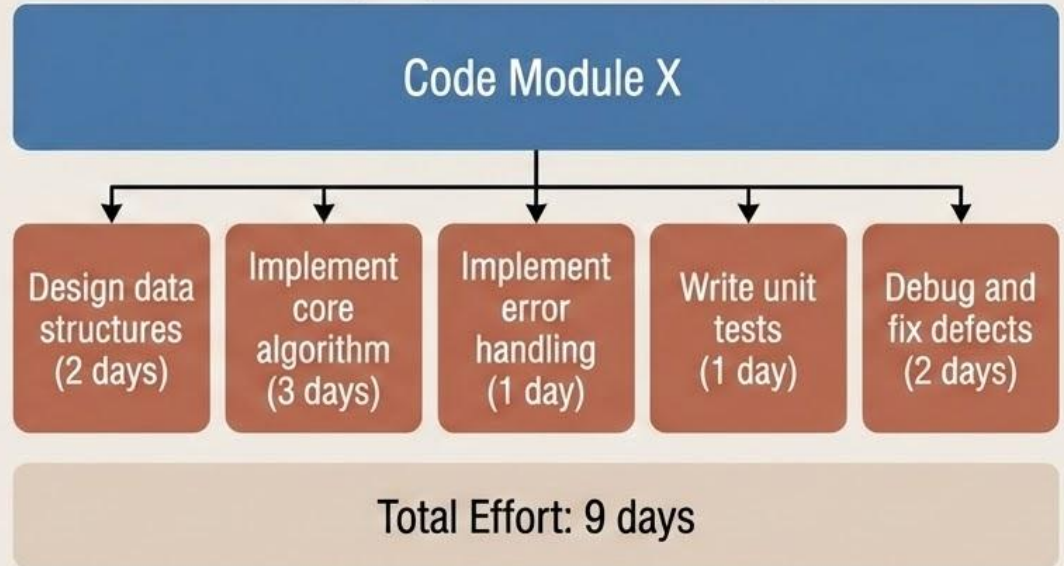
The Challenge:

High-level tasks (e.g., “Code Module X”) are too vague to schedule accurately or assign to a developer.

The Solution:

Refinement. We must break high-level tasks into detailed, granular subtasks with specific effort estimates.

Refinement Example (“Code Module X”)



Mapping the Flow

Defining a Task Network

Visualizing the Sequence of Work

What is a Task Network?

Often called a PERT or CPM (Critical Path Method) chart, it is a graphical representation of the sequence and dependencies of your project tasks.

Core Elements:

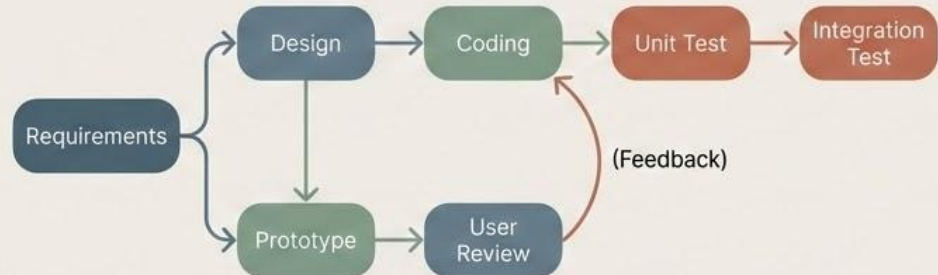


Nodes: Represent the individual Tasks or Milestones.



Arrows: Represent the Dependencies (the required sequence) between tasks.

Example Task Flow:



Task Dependency Types

The Rules of the Network

Tasks are linked by strict rules that dictate when they can start or end relative to one another.

Finish-to-Start (FS)

Task B cannot start until Task A completely finishes.

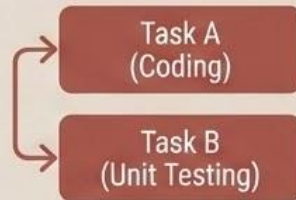
Real-World Example:
Design must finish before Coding can start.



Start-to-Start (SS)

Task B cannot start until Task A starts (they run in parallel).

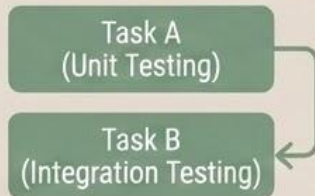
Real-World Example:
Coding starts, which allows Unit Testing to start.



Finish-to-Finish (FF)

Task B cannot finish until Task A finishes.

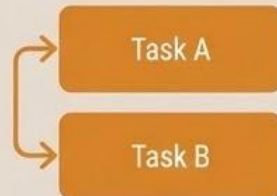
Example: Unit Testing cannot officially finish until Integration Testing finishes.



Start-to-Finish (SF)

Task B cannot finish until Task A starts.

Example:
(Very rare in software engineering).



Visualizing the Timeline

Time-Line Charts (Gantt Charts)

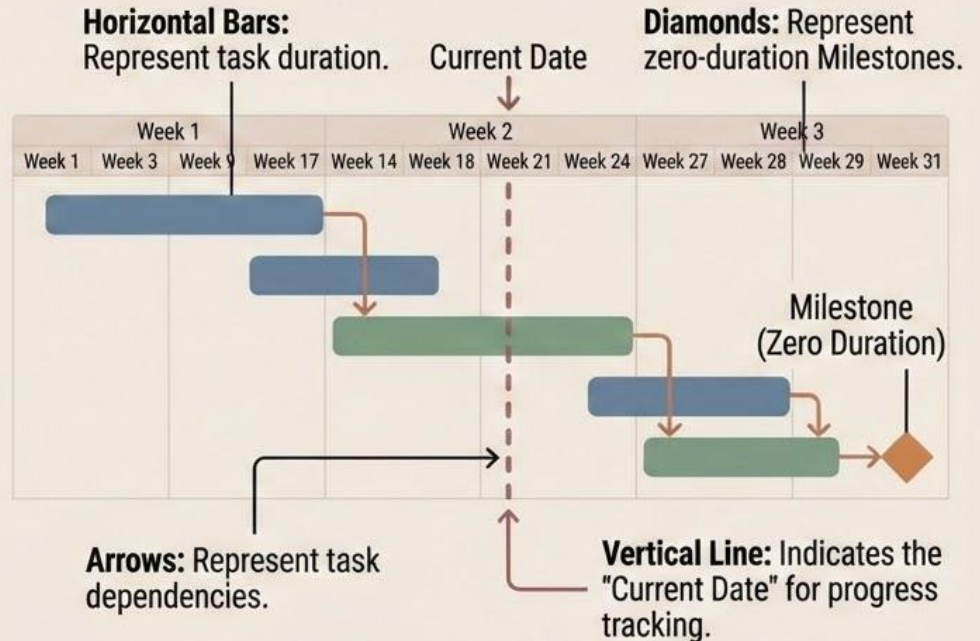
Mapping Tasks to the Calendar

Definition: A horizontal bar chart showing tasks, durations, and dependencies over time.

Origins: Invented by Henry Gantt in the 1910s.

The Trade-off:

- **Pros:** Incredibly easy for stakeholders to understand visually.
- **Cons:** Can struggle to clearly show complex dependencies on very large projects.



Tracking the Schedule

Proactive Project Management

A schedule is useless if you don't track against it.

Tracking Methods

- **Weekly Status Reports:** Teams report actual vs. planned progress.
- **Milestone Tracking:** Binary checks—is the milestone complete or not?
- **Slip Tracking:** Identifying tasks that are falling behind their scheduled end dates.
- **Earned Value Analysis:** A quantitative, mathematical measurement of progress (covered in Part 4).

The “One-Day” Rule

If a task slips by even one day, take action immediately. Small slips accumulate into massive project delays.

The Tracking Dashboard

- **Green:** On track.
- **Yellow:** At risk (likely to slip).
- **Red:** Behind schedule (needs immediate intervention).

Tracking Object-Oriented Projects

Measuring Structural Progress

When tracking an Object-Oriented (OO) project, you don't just track raw "coding hours". You track structural milestones.

OO-Specific Tracking Metrics

- Number of classes identified/designed vs. planned.
- Number of classes implemented vs. planned.
- Number of methods tested vs. planned.

Example Variance Report

Milestone Phase	Planned	Actual	Variance (Risk)
Classes Identified	35	32	-3
Classes Designed	35	28	-7
Classes Coded	35	15	-20 (High Risk)
Classes Tested	35	8	-27 (Critical)

Scheduling for WebApp Projects

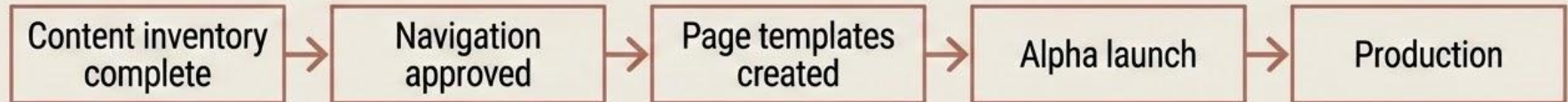
Rapid Iteration and Continuous Delivery



WebApp Characteristics ➔ Content-heavy, rapid iteration, constantly changing requirements.

Scheduling Approaches		Description	Best For
Time-Boxed	Fixed iteration lengths (e.g., rigid 2-week sprints).	Agile WebApp projects.	
Feature-Based	Scheduling by completed function, regardless of phase.	Content-heavy, modular sites.	
Rolling Wave	Detailed scheduling for the near-term; high-level estimates for the future.	Projects with highly volatile, changing requirements.	

Key WebApp Milestones



Measuring True Progress

Earned Value Analysis (EVA)

Quantifying Project Health

Definition:

A quantitative technique for measuring actual project progress and mathematically predicting future performance.

The Core Questions:

- ➔ How much work has been completed?
- ➔ What is the financial value of that completed work?

The Three Fundamental Metrics:

PV (Planned Value / BCWS)

The budgeted cost of the work that was scheduled to be done by today.

EV (Earned Value / BCWP)

The budgeted cost of the work actually completed by today.

AC (Actual Cost / ACWP)

The actual money spent to accomplish the work completed by today.

Are We on Track?

Performance Indices & Variance: Decoding the Metrics

Scenario: We are at Week 4

 **PV:** \$50,000 (Planned Value - 50% complete target)

 **EV:** \$40,000 (Earned Value - Actual progress)

 **AC:** \$45,000 (Actual Cost - Spent)

1. Schedule Performance: Are we on time?

Schedule Variance (SV):

$$\begin{aligned}\$SV &= EV - PV\$ \\ &\rightarrow \$40,000 - 50,000 \\ &= \text{\textcolor{red}{-\$10,000}}\end{aligned}$$



(Negative = **Behind Schedule**)

Schedule Performance Index (SPI):

$$\begin{aligned}\$SPI &= EV / PV\$ \\ &\rightarrow \$40,000 / 50,000 \\ &= \text{\textcolor{red}{0.80}}\end{aligned}$$



(< 1.0 = **Behind Schedule**)

2. Cost Performance: Are we on budget?

Cost Variance (CV):

$$\begin{aligned}\$CV &= EV - AC\$ \\ &\rightarrow \$40,000 - 45,000 \\ &= \text{\textcolor{red}{-\$5,000}}\end{aligned}$$



(Negative = **Over Budget**)

Cost Performance Index (CPI):

$$\begin{aligned}\$CPI &= EV / AC\$ \\ &\rightarrow \$40,000 / 45,000 \\ &= \text{\textcolor{red}{0.89}}\end{aligned}$$



(< 1.0 = **Over Budget**)

Forecasting the Future

Predictive Metrics: Where Will We End Up?

If current performance trends continue, how will the project finish?

BAC (Budget at Completion)



The original total budget for the project.
(Assume \$200,000)

EAC (Estimate at Completion)



What the project will actually cost when finished.

$$EAC = BAC / CPI \rightarrow \$200,000 / 0.89 = \textbf{\$224,719}$$

ETC (Estimate to Complete)



How much more money we need from today to finish.

$$ETC = EAC - AC \rightarrow \$224,719 - 45,000 \\ = \textbf{\$179,719}$$

VAC (Variance at Completion)



How far over/under budget we will be at the very end.

$$VAC = BAC - EAC \rightarrow \$200,000 - 224,719 \\ = \textbf{-\$24,719}$$

In-Class Exercise

Calculate the Health

The Scenario



You are auditing a software project at Week 8.



BAC: \$100,000



PV: \$60,000



EV: \$50,000



AC: \$55,000

Your Task (5 Minutes)



- Calculate the Schedule Performance Index (SPI). Is it on schedule?
- Calculate the Cost Performance Index (CPI). Is it on budget?
- Calculate the Estimate at Completion (EAC). What will the final cost be?